

COMSATS UNIVERSITY ISLAMABAD

Department of Computer Science (BSCS)

FIREWALL RULE MANAGEMENT SYSTEM (FRMS)

A Cybersecurity Simulation Using Data Structures

Course	CSC211 — Data Structures
Semester	Spring 2026
Teacher	Ms. Najla Raza
Submission Date	May 6, 2026

Project Report Submitted by:

#	Full Name	Registration No.
1	M. Monis Raza	SP25-BCT-032
2	Muhammad Zeeshan Iqbal	SP25-BCT-037
3	Syed Attaullah Shah	SP25-BCT-045

1. Executive Summary

The Firewall Rule Management System (FRMS) is a fully functional cybersecurity simulation developed in C++ as a semester project for CSC211 — Data Structures at COMSATS University Islamabad. The project simulates how a real-world firewall engine processes incoming network packets, matches them against predefined security rules, and produces an auditable log of all decisions made.

The system was designed to demonstrate the practical application of five distinct data structures — Queue, Hash Table, Max-Heap, Binary Search Tree (BST), and Array — and four algorithms — Hashing, Quick Sort, Merge Sort, and Binary Search — in a cohesive, working software system. Each data structure was selected based on its computational strengths and direct mapping to a real firewall subsystem.

The project is entirely object-oriented, applying all four pillars of OOP: Encapsulation, Abstraction, Inheritance, and Polymorphism. It features a menu-driven console interface, file I/O for rules and packets, real-time packet processing, log sorting, and a statistical summary dashboard.

2. Problem Statement

In today's digital landscape, network security is a foundational concern for organizations of all sizes. A firewall is the first line of defense — it inspects every packet entering or leaving a network and decides, based on security rules, whether to allow or block it. Real-world firewalls face immense computational pressure:

- Millions of packets must be processed every day at high speed.
- Each packet must be checked against potentially hundreds of security rules.
- Rules carry different priority levels — a critical block rule must override a general allow rule.
- All decisions must be logged and remain searchable for forensic analysis.
- IP-based blocking must extend to ranges, not just individual addresses.

Without appropriate data structures, a naive implementation of these requirements would be:

- Slow — $O(n)$ linear scanning through every rule for every packet.
- Unordered — no priority enforcement; dangerous packets could slip through.
- Unscalable — performance degrades catastrophically as rule sets grow.
- Unanalyzable — logs stored arbitrarily cannot be searched or sorted efficiently.

This project addresses each of these challenges by mapping the right data structure to each problem, simulating how production-grade security software is designed and optimized.

3. Project Objectives

1. Simulate a fully functional firewall engine in software using C++.
2. Demonstrate how a Queue handles incoming packets in correct FIFO order.
3. Demonstrate $O(1)$ rule lookup using a custom-built Hash Table with chaining.

- 4. Apply a Max-Heap to enforce rule priority — ensuring critical rules are always evaluated first.
- 5. Use a Binary Search Tree to manage IP-based blocking with $O(\log n)$ insertion and lookup.
- 6. Apply Quick Sort and Merge Sort to organize security logs.
- 7. Implement Binary Search to retrieve log entries by IP address efficiently.
- 8. Apply all four OOP principles: Encapsulation, Abstraction, Inheritance, and Polymorphism.
- 9. Support file-based input/output for rules, packets, and logs.
- 10. Provide a user-friendly, menu-driven console interface.

4. Real-World Relevance

This project is directly inspired by real production cybersecurity systems. Every component of FRMS has a counterpart in the industry:

Real-World System	Examples	Our FRMS Component
Corporate Firewalls	Cisco ASA, Palo Alto	HashTable + Rule Engine
Intrusion Detection Systems	Snort, Suricata	MaxHeap + BST
Packet Processing Engines	Linux iptables, nftables	PacketQueue (FIFO)
SIEM / Log Analysis	Splunk, IBM QRadar	LogManager + Sort Algorithms
IP Blacklisting Services	Cloudflare, Spamhaus	BST + HashTable

5. System Architecture & OOP Design

The system is built around five classes that together form a complete, layered firewall engine. OOP principles govern the entire design.

5.1 Class Structure

Class	Key Attributes	Key Methods
Packet	sourceIP, destIP, port, protocol, size	getSourceIP(), getPort(), display()
Rule (base)	ruleID, targetIP, action, priority, hitCount	getAction(), getPriority(), operator>(), display()
BlockRule : Rule	Inherits Rule, action = BLOCK	display() [overridden]
AllowRule : Rule	Inherits Rule, action = ALLOW	display() [overridden]

LogEntry	timestamp, action, sourceIP, port, ruleID, priority	display()
Firewall	packetQueue, ruleTable, ruleHeap, ipTree, logManager	loadRules(), loadPackets(), processAllPackets(), addRule(), removeRule(), saveLogs()

5.2 OOP Principles Applied

Encapsulation

Every class declares its data members as private. External code can only interact through well-defined public getter/setter methods. For example, the Packet class stores sourceIP, destIP, port, protocol, and size privately, exposing them only via getSourceIP(), getPort(), etc. This prevents accidental data corruption and keeps internal logic hidden.

Abstraction

The Firewall class acts as a high-level controller. A user of the system only calls fw.processAllPackets() — without knowing whether the engine internally uses a Hash Table, a Heap, or a BST. The underlying complexity is completely hidden behind a simple, clean interface.

Inheritance

The Rule class is the base class. BlockRule and AllowRule both inherit from Rule. This means common attributes (ruleID, targetIP, priority, hitCount) and behaviors are defined once in Rule, while each subclass only defines what makes it unique — the display() override and the hardcoded action value.

Polymorphism

Both BlockRule and AllowRule override the virtual display() method from the base Rule class. When a base-class Rule pointer points to a BlockRule object, calling display() correctly invokes the BlockRule version, printing "[BLOCK]" prefixes. This is runtime polymorphism via virtual function dispatch.

6. Data Structures — Detailed Explanation

6.1 Queue — FIFO Packet Processing (Linear)

Category: Linear | Assigned to: M. Monis Raza (SP25-BCT-032)

The PacketQueue is a custom implementation using a circular array of MAX_PACKETS (500) slots. It models the arrival buffer of a real network interface — packets arrive in order and must be processed in that exact order.

Operation	Complexity	Description
push (packet)	$O(1)$	Adds new packet to the rear of the circular array

<code>pop()</code>	$O(1)$	Removes and returns packet from the front
<code>front()</code>	$O(1)$	Peeks at the front packet without removing it
<code>empty()</code>	$O(1)$	Returns true if no packets are queued

Justification: Without a Queue, packets could be processed out of order. In a security context, this can open race conditions where a connection handshake packet is evaluated after the data packet, causing incorrect decisions.

6.2 Hash Table — Rule Lookup (Non-Linear)

Category: Non-Linear | Assigned to: M. Zeeshan Iqbal (SP25-BCT-037)

The HashTable was built entirely from scratch — without using STL's `unordered_map`. It uses an array of 101 buckets (a prime number chosen for optimal distribution) and resolves collisions via chaining (linked lists at each bucket).

The hashing algorithm uses a polynomial rolling hash on the IP address string. Each character of the IP is multiplied by a prime multiplier (31) and accumulated modulo 101. This produces a well-distributed index for typical IP address inputs.

```
hash = 0, multiplier = 1, prime = 31
for each character c in IP:
    hash = (hash + (c - ' ' + 1) * multiplier) % TABLE_SIZE
    multiplier = (multiplier * prime) % TABLE_SIZE
return hash
```

Operation	Avg. Complexity	Description
<code>insert(ip, rule)</code>	$O(1)$	Hashes IP, stores rule in bucket
<code>search(ip)</code>	$O(1)$	Returns pointer to rule, or nullptr
<code>remove(ip)</code>	$O(1)$	Unlinks node from chain, frees memory
<code>displayAll()</code>	$O(n)$	Prints all rules across all buckets

Justification: Without a Hash Table, every packet would require scanning all rules in sequence — $O(n)$ per packet. For 1,000 packets and 200 rules, that is 200,000 comparisons. With the Hash Table, each lookup is a single hash computation and at most a few chained comparisons.

6.3 Max-Heap — Rule Priority Management (Non-Linear)

Category: Non-Linear | Assigned to: Syed Attaullah Shah (SP25-BCT-045)

The MaxHeap stores all firewall rules ordered by their priority (1 = lowest, 10 = highest). Using a max-heap guarantees that the highest-priority rule is always at the root and can be accessed in $O(1)$ time. This models real firewalls where a rule blocking a known malicious IP (priority 10) must be enforced before a general "allow" rule (priority 2). The heap is maintained via the heapify-up operation on insert and heapify-down on pop.

Operation	Complexity	Description
<code>push(rule)</code>	$O(\log n)$	Insert + heapify-up
<code>top()</code>	$O(1)$	Peek at highest-priority rule
<code>pop()</code>	$O(\log n)$	Remove root + heapify-down
<code>size()</code>	$O(1)$	Return number of rules in heap

6.4 Binary Search Tree — IP Range Blocking (Non-Linear)

Category: Non-Linear | Assigned to: Syed Attaullah Shah (SP25-BCT-045)

The BST maintains all BLOCK-rule IP addresses in a sorted tree structure. Each node stores an IP address. Left subtrees hold lexicographically smaller IPs; right subtrees hold larger ones. The inorder traversal of the BST produces all blocked IPs in sorted order.

When the Hash Table finds no exact match for a packet's source IP, the BST is queried. This provides a fallback mechanism for IP range blocking and an independent secondary check.

Operation	Complexity	Description
<code>insert(ip)</code>	$O(\log n)$	Insert blocked IP maintaining BST property
<code>isBlocked(ip)</code>	$O(\log n)$	Returns true if IP found in tree
<code>inorder()</code>	$O(n)$	Prints all blocked IPs in sorted order

6.5 Array — Log Storage (Linear)

Category: Linear | Assigned to: M. Monis Raza (SP25-BCT-032)

The LogManager stores all firewall decision records in a static array ($\text{MAX_LOGS} = 1000$). Each element is a LogEntry containing timestamp, action (BLOCKED/ALLOWED), source IP, port, rule ID, and priority. The array supports index-based access which is essential for the sorting algorithms to work in-place.

7. Algorithms — Detailed Explanation

7.1 Hashing Algorithm

Used in the Hash Table to convert an IP address string to a bucket index. A polynomial rolling hash is used — each character position contributes a different weight based on powers of 31. This produces significantly fewer collisions than simple character-sum hashing.

Complexity: $O(k)$ where k = length of IP string (constant in practice, at most 15 characters).

7.2 Quick Sort — Log Sorting by Timestamp

Assigned to: M. Monis Raza (SP25-BCT-032)

Quick Sort is applied to the log array when the user requests sorting by timestamp. It uses a pivot-based partition scheme — all entries with timestamps before the pivot go to the left partition, and those after go to the right. The algorithm recurses on both halves.

- Average Complexity: $O(n \log n)$
- Worst Case: $O(n^2)$ — avoided with good pivot selection
- Space: $O(\log n)$ stack space for recursion
- In-place sorting — no additional array needed

7.3 Merge Sort — Log Sorting by Severity

Assigned to: M. Monis Raza (SP25-BCT-032)

Merge Sort is applied when the user requests logs sorted by severity (rule priority). It was chosen over Quick Sort for this specific task because it is a stable sort — logs with equal priority retain their original chronological order.

- Complexity: $O(n \log n)$ — guaranteed in all cases
- Stable: logs with equal priority stay in arrival order
- Splits array into halves, sorts each half, then merges

7.4 Binary Search — Log Lookup by IP

Assigned to: M. Monis Raza (SP25-BCT-032)

Once the log array is sorted (by IP address), Binary Search allows $O(\log n)$ lookup of all log entries for a specific source IP — much faster than a linear scan across thousands of log records.

- Complexity: $O(\log n)$ on a sorted array
- Requires prior sorting of the log array by IP

8. System Workflow

The packet processing pipeline follows a strict 4-step decision process for each dequeued packet:

```
START
Load Rules —> Hash Table + Max-Heap + BST
```

```

Load Packets -> Enqueue into PacketQueue (FIFO)
LOOP: Dequeue packet from front of Queue
  STEP 1: Hash Table lookup (exact IP match)          O(1)
  STEP 2: BST search (secondary IP block check)       O(log n)
  STEP 3: Max-Heap (priority-ordered fallback check)  O(log n)
  STEP 4: Default Policy = ALLOW (if no rule found)
Log Decision -> LogManager array
REPEAT until Queue empty
END

```

This layered approach mirrors how real firewalls chain rule evaluation: fast exact matches first, slower range checks second, and a safe default last.

9. System Features & Console Interface

FRMS ships with a menu-driven console interface offering 15 operations accessible at runtime:

#	Menu Option	What It Does
1	Load Rules & Packets	Loads rules from rules.txt + 7 hardcoded defaults into all 3 data structures; loads packets into Queue
2	Process All Packets	Dequeues and evaluates every packet; prints decision for each; logs all results
3	Add a New Rule	Prompts for rule details; creates BlockRule or AllowRule; inserts into HashTable, Heap, and BST
4	Remove a Rule (by IP)	Removes a rule from the Hash Table by target IP
5	Add Packet Manually	Prompts for packet details; enqueues into PacketQueue for next run
6	View Firewall Logs	Displays all logged decisions in tabular format
7	Sort Logs by Timestamp	Applies Quick Sort to log array; re-displays sorted
8	Sort Logs by Severity	Applies Merge Sort by priority; highest severity first
9	Search Log by IP	Searches log entries for a given source IP
10	View Hash Table	Dumps all rules stored in the Hash Table with their bucket positions
11	View Max-Heap	Displays all rules in priority order
12	View BST (Blocked IPs)	Inorder traversal — lists all blocked IPs in sorted order
13	Save Logs to File	Writes all log entries to data/logfile.txt with summary statistics
14	Display Summary	Shows total processed, blocked, allowed, and block rate %

0

Exit

Gracefully shuts down the system

10. Input & Output Specification

10.1 Input: rules.txt

The system reads firewall rules from a comma-separated file. The header row is skipped; subsequent rows are parsed and loaded into all three data structures simultaneously.

RuleID	TargetIP	Action	Priority	Protocol
R001	192.168.1.10	BLOCK	9	ANY
R002	8.8.8.8	ALLOW	5	UDP
R003	172.16.0.1	BLOCK	8	TCP
R004	45.33.32.156	BLOCK	10	HTTP
R005	10.0.0.5	ALLOW	3	TCP
R006	203.0.113.50	BLOCK	7	ANY
R007	192.168.1.50	BLOCK	6	ANY

10.2 Input: packets.txt

Incoming network packets are loaded from a CSV file with five fields. Each parsed packet is enqueued into the PacketQueue.

SrcIP	DestIP	Port	Protocol	Size
192.168.1.10	10.0.0.5	8080	TCP	1024
8.8.8.8	192.168.1.1	53	UDP	256
172.16.0.1	10.0.0.1	443	TCP	512
45.33.32.156	192.168.0.1	80	HTTP	2048
10.0.0.5	8.8.8.8	443	TCP	768
203.0.113.50	10.0.0.2	22	ANY	128
192.168.1.50	172.16.0.5	3306	ANY	4096
1.1.1.1	10.0.0.1	80	HTTP	300

10.3 Output: logfile.txt

After processing, all decisions are written to data/logfile.txt in a formatted table, followed by a summary line showing totals, block count, allow count, and block rate percentage.

Timestamp	Action	SourceIP	Port	RuleID	Priority
2026-05-06 14:22:01	BLOCKED	192.168.1.10	8080	R001	9
2026-05-06 14:22:01	ALLOWED	8.8.8.8	53	R002	5
2026-05-06 14:22:01	BLOCKED	172.16.0.1	443	R003	8
2026-05-06 14:22:01	BLOCKED	45.33.32.156	80	R004	10
2026-05-06 14:22:01	ALLOWED	10.0.0.5	443	R005	3
2026-05-06 14:22:01	BLOCKED	203.0.113.50	22	R006	7
2026-05-06 14:22:01	BLOCKED	192.168.1.50	3306	R007	6
2026-05-06 14:22:01	ALLOWED	1.1.1.1	80	DEFAULT	0

 Total: 8 packets | Blocked: 6 | Allowed: 2 | Block Rate: 75%

11. Team Division of Work

The project was divided into three focused modules, each owned end-to-end by one team member — from design to implementation to testing. All members collaborated on integration, testing, and documentation.

Team Member	Reg. No.	Assigned Modules & Responsibilities
M. Monis Raza	SP25-BCT-032	- PacketQueue class — circular array implementation, push/pop/front/empty operations - LogEntry and LogManager classes — array-based log storage, addLog(), displayAll(), writeToFile() - Quick Sort algorithm — sort logs by timestamp - Merge Sort algorithm — sort logs by severity/priority - Binary Search — search logs by IP address - File I/O for packets.txt and logfile.txt
M. Zeeshan Iqbal	SP25-BCT-037	- HashTable class — built from scratch with chaining collision resolution - Polynomial rolling hash function on IP address strings - HashNode struct — linked list chaining within each bucket - Rule base class + BlockRule and AllowRule derived classes - Operator overloading (> and <) on Rule for Heap comparisons - File I/O for rules.txt parsing with whitespace trimming and error handling
Syed Attaullah Shah	SP25-BCT-045	- MaxHeap class — array-based max-heap with heapify-up and heapify-down - BST class — binary search tree for blocked IP management (insert, isBlocked, inorder) - Firewall main controller class — orchestrates all data structures - processPacket() — 4-step decision pipeline (Hash Table → BST → Heap → Default) - Console UI — printBanner(), printMenu(), and full main() switch-case loop - Full system integration, testing, and debugging

12. Course Learning Outcomes (CLO) Mapping

The project covers all major CLO 5 requirements of CSC211:

Data Structure / Algorithm	Category	Complexity	CLO 5 Coverage
Queue (PacketQueue)	Linear	$O(1)$ push/pop	FIFO packet processing order
Array (LogManager)	Linear	$O(1)$ access	Sequential log storage + sort
Hash Table	Non-Linear	$O(1)$ avg search	$O(1)$ rule lookup by IP
Max-Heap	Non-Linear	$O(\log n)$ push/pop	Priority-based rule management
BST	Non-Linear	$O(\log n)$ search	IP range-based blocking
Quick Sort	Algorithm	$O(n \log n)$ avg	Sort logs by timestamp
Merge Sort	Algorithm	$O(n \log n)$ stable	Sort logs by severity
Binary Search	Algorithm	$O(\log n)$	Find log by IP efficiently

13. Project Timeline

Week	Timeline	Activities
Week 1	Early April 2026	Project setup, requirement analysis, class design, division of work, proposal writing
Week 2	Mid April 2026	Core data structure implementation: PacketQueue (Monis), HashTable (Zeeshan), MaxHeap + BST (Attaullah)
Week 3	Late April 2026	Sorting algorithms integration, Firewall controller class, Console UI, File I/O, system integration
Week 4	May 1–6, 2026	Full system testing, debugging edge cases, documentation, report writing, final submission

14. Tools & Technologies Used

Tool / Technology	Purpose
C++ (ISO C++11/14)	Primary programming language — classes, OOP, file I/O, STL-free data structures
Visual Studio Code	Primary IDE for writing and editing source code
GCC Compiler (g++)	Compilation and linking of the C++ source file
GitHub	Version control — tracking changes, collaboration between 3 team members
MS Word	Documentation, proposal writing, and report preparation
Data Files (.txt)	Input/output — rules.txt, packets.txt, logfile.txt

15. Challenges Faced & Solutions

- **Challenge:** Building a Hash Table from scratch without STL.

Solution: Implemented polynomial rolling hash with chaining. Chose 101 as table size (prime) for better distribution. Carefully managed memory with `delete[]` in destructor.

- **Challenge:** Heap ordering required custom comparison on Rule objects.

Solution: Overloaded `operator>` and `operator<` in the Rule class, enabling the MaxHeap to compare rules by priority without knowing internal details.

- **Challenge:** Sorting logs by two different criteria (timestamp vs. severity).

Solution: Used two separate algorithms — Quick Sort for timestamps (fastest general case) and Merge Sort for severity (stability required).

- **Challenge:** Integrating three independent data structures (HashTable, Heap, BST) into one coherent decision pipeline.

Solution: Designed the Firewall class as a single controller composing all three, with a clean 4-step `processPacket()` method as the integration point.

- **Challenge:** Handling malformed or partially empty file lines gracefully.

Solution: Added whitespace trimming lambda and try-catch blocks around `stoi()` calls to skip bad lines with warning messages without crashing.

16. Conclusion

The Firewall Rule Management System is a complete, functional, and well-structured cybersecurity simulation that successfully demonstrates the real-world application of five data structures and four algorithms within a single cohesive project.

Each design decision was driven by algorithmic reasoning: the Queue was chosen for its FIFO guarantee; the Hash Table for its $O(1)$ lookup speed; the Max-Heap for priority enforcement; the BST for structured IP management; and the sorted Array for efficient log analysis. The system is fully object-oriented, using all four OOP pillars effectively.

Beyond meeting the academic requirements of CSC211, this project builds genuine software engineering skills. Designing a multi-module system, managing data structures concurrently, handling file I/O with error recovery, and writing clean, documented C++ code are skills directly applicable in industry — particularly in systems programming and cybersecurity software development.

The team successfully divided responsibilities, integrated independently developed modules, and delivered a working, tested system within the four-week timeline.

17. References

11. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
12. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
13. Cisco Systems. (2024). Cisco ASA Firewall Configuration Guide. Cisco Press.
14. Linux Man Pages. iptables(8) — administration tool for IPv4 packet filtering. linux.die.net
15. Knuth, D. E. (1998). The Art of Computer Programming, Vol. 3: Sorting and Searching. Addison-Wesley.
16. CSC211 Lecture Slides — Data Structures. COMSATS University Islamabad, Spring 2026. Instructor: Ms. Najla Raza.

—— *End of Report* ——

Firewall Rule Management System (FRMS) | CSC211 Data Structures | Spring 2026